FICHA DE TRABAJO 02

Evaluación de una aplicación móvil: usabilidad, funcionalidad, límites y estrés

Dato	Información
Asignatura	Desarrollo de Aplicaciones Móviles I — sexto semestre
Aplicación	Casa Familias
Integrante 1	_____
Integrante 2	_____
Fecha de entrega	_____
Versión de referencia	4.2, código 3. Se adjunta para comparación de interfaz y limpieza.
Versión mejorada	4.3.1, código 5. Android 8.0 o posterior; APK de pruebas.

1. Propósito y alcance

Evaluar si Casa Familias permite consultar pagos y deudas, administrar el QR, editar familias, reasignar usuarios y organizar la limpieza semanal. El proceso seguido es: probar, registrar, analizar, corregir y volver a probar.

La evidencia técnica se obtuvo con pruebas automáticas locales, Chromium con datos simulados y PostgreSQL desechable. Estas pruebas no equivalen a una evaluación con personas ni prueban el rendimiento del teléfono o del servidor real. No se modificaron registros del Supabase real para llenar esta ficha.

Tecnología real: proyecto Android Studio con contenedor Java, interfaz HTML/CSS/JavaScript y Supabase. La relación con Kotlin de la sección 14 es conceptual; este informe no presenta el proyecto como una implementación en Kotlin.

2. Verificación inicial de la versión mejorada

Verificación	Sí / No	Observaciones
Abre correctamente	Sí, local	Primer inicio y sesión simulada comprobados. En el teléfono falta confirmar el inicio después de conectarse a internet.
La navegación funciona	Sí, local	26 vistas de demostración; formularios y permisos por rol.
Operaciones principales	Sí, local	QR, nombres, asignaciones y limpieza verificados con backend simulado o SQL aislado.
Procesamiento de datos	Sí, local	Cálculo de Restaurante y reglas SQL verificadas; no es conciliación de pagos reales.
Existen validaciones	Sí	Formato/tamaño del QR, nombre, permisos y duplicados.
Mensajes comprensibles	Por confirmar	Hay mensajes específicos; su comprensión debe observarse con un compañero.
La información se conserva	Parcial	Persistencia SQL y almacenamiento local comprobados; cerrar/reabrir el APK con datos reales queda pendiente.

3. Fase 1 — Qué vamos a probar

Entregar únicamente el enunciado de la tarea. Usar cuentas y datos de prueba con el rol indicado. No registrar pagos bancarios ficticios como si fueran reales.

N.º / rol	Tarea para entregar al participante	Resultado esperado
T1 · Familia	Ingresa con tu cuenta y averigua cuánto debes.	Accede al inicio y reconoce el saldo de su familia.
T2 · Familia	Encuentra el QR de pago e identifica cómo enviarías el comprobante.	Localiza el QR, las instrucciones y el formulario; no hace falta transferir dinero.
T3 · Administrador	Cambia el nombre de una familia de prueba.	Guarda el nuevo nombre y lo ve actualizado.
T4 · Administrador	Asigna una cuenta de prueba a otra familia.	La nueva asignación se guarda y aparece al actualizar.
T5 · Administrador	Revisa el orden de limpieza e identifica cómo repetirlo por semana.	Reconoce Choque, Galarza, Salinas, David y Acha; Salinas2 comparte el turno de Salinas.

4. Fase 2 — Observación de usabilidad

Participante: _____ Rol probado: _____ Dispositivo: _____. Observar sin indicar botones ni rutas. Si hay riesgo de alterar datos reales, detener la prueba y registrarlo.

Aspecto observado	Observación a completar durante la sesión
¿Encontró dónde comenzar?	Pendiente: anotar la primera acción.
¿Comprendió qué debía hacer?	Pendiente: anotar dudas sobre el objetivo.
¿Entendió textos y botones?	Pendiente: copiar el texto que causó confusión.
¿Dudó o se equivocó?	Pendiente: registrar el paso y la cantidad de errores.
¿Necesitó ayuda?	Pendiente: registrar cuántas veces y qué ayuda recibió.
¿Encontró cómo regresar?	Pendiente: observar navegación, Cerrar y Atrás del teléfono.
¿Comprendió los errores?	Pendiente: pedir que explique el mensaje con sus palabras.
¿Reconoció que terminó?	Pendiente: observar si identifica la confirmación.
¿Completó la tarea?	Pendiente: comparar con el resultado esperado.

5. Medición de la usabilidad

Tarea	Tiempo (s)	Errores	Ayudas	¿Completó?
T1	____	____	____	Sí [ ] No [ ]
T2	____	____	____	Sí [ ] No [ ]
T3	____	____	____	Sí [ ] No [ ]
T4	____	____	____	Sí [ ] No [ ]
T5	____	____	____	Sí [ ] No [ ]

Pendiente de observación. Cronometrar desde la entrega de la tarea hasta su finalización o abandono. Definir antes qué cuenta como error y ayuda. Éxito = completadas / intentadas × 100. Comparar versiones bajo condiciones equivalentes.

6. Fase 3 — Diez casos funcionales

Resultado «Correcto (local)»: pasó una comprobación automatizada. En la columna Obtenido se indica el entorno. No significa que un compañero haya completado la tarea ni que se haya probado una transferencia real.

ID	Función / entrada o pasos	Resultado esperado	Resultado obtenido	Estado
CP-01	Primer inicio sin sesión; abrir los scripts en su orden real.	Mostrar el formulario de ingreso.	Correcto en prueba del parser y Chromium con SDK incluido.	Correcto (local)
CP-02	Recuperación: introducir contraseñas distintas.	Rechazar la confirmación sin actualizar la contraseña.	La prueba de autenticación conserva el formulario y muestra el error.	Correcto (local)
CP-03	Administrador: guardar una imagen QR admitida.	Subirla y guardar su referencia.	Subida y RPC simuladas; la referencia se actualiza. Imagen bancaria real pendiente.	Correcto (simulado)
CP-04	QR: simular fallo «Bucket not found».	Explicar la falta de configuración y permitir reintento.	Mensaje de almacenamiento; conserva título y vista previa.	Correcto (simulado)
CP-05	Administrador: renombrar una familia de prueba.	Guardar el nombre sin reemplazar su identidad.	Nombre actualizado en interfaz simulada y conservado en SQL aislado.	Correcto (local)
CP-06	Administrador: reasignar una cuenta de familia.	Actualizar su familia y permitir verla al refrescar.	La asignación cambia; se conserva el rol y se validan los identificadores.	Correcto (local)
CP-07	Familia, encargado y Restaurante: intentar funciones de administrador.	Impedir cambios reservados al administrador.	La interfaz bloquea los formularios; SQL comprueba permisos del servidor.	Correcto (local)
CP-08	Guardar ciclo semanal con cinco familias únicas.	Repetir el orden sin crear dos turnos para una misma semana.	Interfaz de cinco posiciones; SQL verifica repetición e idempotencia en fixtures.	Correcto (local)
CP-09	Cuenta Salinas2: consultar y completar el turno compartido.	Compartir turno con Salinas y mantener la separación contable.	Alias y permisos comprobados con familias ficticias; interfaz habilita completar.	Correcto (local)
CP-10	Restaurante: deuda 100 y pago 40.	Mostrar saldo 60 y navegación de consulta.	El test de Restaurante devuelve 60 y verifica restricciones de navegación.	Correcto (local)

Fuentes: pruebas-interfaz.txt; pruebas-base-datos.txt; test_auth.cjs, test_admin.cjs, test_rotation.cjs, test_restaurant.cjs y los archivos SQL de validación del proyecto. Las cuentas y registros de estos casos son ficticios.

7. Fase 4 — Pruebas de validación

Caso	Entrada incorrecta	Resultado esperado	Resultado obtenido / estado
V-01	Nombre vacío o solo espacios.	Rechazar y explicar qué falta.	Interfaz controla vacío y SQL rechaza el nombre vacío; correcto local.
V-02	Archivo PDF como imagen del QR.	Aceptar únicamente PNG, JPG o WebP.	Se rechaza antes de subir; correcto con storage simulado.
V-03	Imagen de 5.242.881 bytes.	Rechazar por superar el límite.	No se llama a la subida; correcto en validación de tamaño.
V-04	Nombre de familia ya usado, variando mayúsculas.	Evitar nombres duplicados.	SQL devuelve rechazo; correcto en PostgreSQL aislado.
V-05	Encargado con la misma familia en ambas asignaciones.	Exigir dos familias diferentes.	Interfaz y SQL lo rechazan; correcto local.
V-06	Usuario de familia intentando cambiar nombres.	Denegar la operación.	Formulario bloqueado y función SQL sin permiso; correcto local.
V-07	Monto vacío, texto «abc», −1 o 0.	Rechazar el ingreso mediante el formulario.	Los cuatro casos fallan checkValidity(); 0,01 pasa. Solo valida las restricciones HTML, no la RPC de pagos.

8. Fase 5 — Pruebas de límites

Cada prueba incluye los dos extremos válidos y los valores inmediatamente fuera. Se ejecutaron 12 comprobaciones: cuatro de tamaño de QR y ocho SQL para nombre y tarea.

Campo	Mínimo válido	Máximo válido	Antes	Después	Resultado
Tamaño de imagen QR	1 byte	5.242.880 bytes (5 MiB)	0 bytes	5.242.881 bytes	Acepta extremos y rechaza exteriores en el control de tamaño.
Nombre de familia	1 carácter	80 caracteres	0 caracteres	81 caracteres	Acepta 1 y 80; rechaza 0 y 81 en SQL.
Texto de una tarea de limpieza	1 carácter	160 caracteres	0 caracteres	161 caracteres	Acepta 1 y 160; rechaza 0 y 161 en SQL.

En nombre y tarea se evalúa la longitud después de quitar espacios de los extremos. La prueba de un archivo de 1 byte solo comprueba el límite de tamaño: no demuestra que ese archivo sea una imagen decodificable o un QR legible. La lista de tareas también tiene un máximo de 40, pero este informe no añade una medición nueva de ese límite.

Evidencia: limites-repeticiones.json, pruebas-limites-sql.txt, limites.sql y validacion-monto.json. El SQL de esta carpeta es exclusivo para una base desechable; no se ejecuta en el proyecto real.

9. Fase 6 — Pruebas de estrés

Pruebas controladas con datos ficticios. No se realizaron ataques ni carga contra Supabase real. Tampoco se midió consumo de memoria, batería ni concurrencia de varios teléfonos.

Prueba	Qué se hizo	Qué se observó	Resultado
E-01 · Cantidad de datos	Renderizar 100, 500 y 1.000 pagos en Chromium, tres veces por nivel.	Cantidad de tarjetas, tiempo hasta dos cuadros de pantalla, errores JavaScript y desbordamiento horizontal.	Se muestran todas las tarjetas, sin errores JavaScript ni desbordamien to en 390 × 844.
E-02 · Operaciones repetitivas	Guardar 20 nombres sucesivos de una familia ficticia en un backend simulado.	Número de solicitudes y último valor conservado.	20 solicitudes; el último nombre es Familia Prueba 19.
E-03 · Acciones rápidas	Enviar 20 intentos de guardar el QR mientras la primera respuesta está retenida.	Solicitudes enviadas y prevención del duplicado simultáneo.	Una solicitud de guardado; 19 intentos adicionales no generan otro guardado.

Mediciones técnicas del renderizado

Registros	Muestras de tiempo (ms)	Mediana (ms)
100	24.5; 17.1; 15.6	17.1
500	72.3; 80.7; 75.4	75.4
1000	130.3; 135; 160.4	135

Las 20 modificaciones secuenciales de E-02 tardaron 579.14 ms en Node.js con jsdom y servidor simulado. Ese tiempo no incluye red, disco del teléfono ni escritura en Supabase real.

Interpretación: al aumentar los registros crece el tiempo de renderizado; la aplicación completó estos niveles en el entorno medido. No se encontró su punto de saturación y no se puede afirmar que admita usuarios o registros ilimitados. No se estableció un umbral de tiempo previo; los valores son descriptivos.

Cómo repetirlo en el teléfono

Usar una base de pruebas con copias ficticias de 100, 500 y 1.000 registros, el mismo dispositivo y la misma conexión. Repetir tres veces cada nivel, registrar duración, bloqueos y memoria si se dispone de herramientas. No completar estos resultados con los tiempos del navegador de escritorio.

Dato del ensayo físico	Registro pendiente
Modelo / Android / WebView	_____
Conexión y condiciones	_____
Bloqueos, lentitud o cierres	_____

10. Fase 7 — Pruebas de persistencia

Los controles SQL comprueban conservación en base de datos y el arranque comprueba que Reintentar no borra un marcador de almacenamiento local. Eso no sustituye las tres pruebas de cerrar y reabrir el APK.

Prueba	Resultado esperado	Obtenido / estado
P-01 · Guardar → cerrar → abrir	Guardar nombre de una familia de prueba, cerrar el APK y comprobarlo al abrir.	Nombre persistido en SQL aislado. Ciclo completo en teléfono: pendiente.
P-02 · Registrar varios → reiniciar → recuperar	Crear tres registros ficticios, reiniciar el teléfono y comprobar que siguen disponibles.	No ejecutado en teléfono. Registrar los identificadores y verificar sin crear duplicados.
P-03 · Modificar → cerrar → abrir → comprobar	Reasignar un usuario de prueba y comprobar su nueva familia tras cerrar y abrir.	Asignación persistida en SQL aislado. Reapertura del APK y cuenta afectada: pendiente.

11. Fase 8 — Detección y clasificación

Tipo	Hallazgo de este trabajo
Funcional	El QR devolvía Bucket not found en una etapa anterior. Faltaba configurar su almacenamiento.
Usabilidad / interfaz	Se solicitaron botones claros, mejor orden visual y ajuste de textos.
Navegación	Se reemplazaron controles con flechas por acciones escritas y botones Cerrar.
Validación	Las pruebas actuales comprueban límites y permisos; no se observó una aceptación indebida en los casos ejecutados.
Persistencia	No se demostró pérdida de datos. La reapertura física completa sigue pendiente.
Rendimiento	La prueba local terminó con 1.000 pagos. La saturación y el teléfono real no se midieron.
Mensaje / conectividad	El usuario comunicó una carga detenida y luego aclaró que estaba sin internet. No hay confirmación posterior de entrada.

12. Fase 9 — Priorización de problemas

Problema	Tipo	Prioridad	Evidencia
QR sin almacenamiento configurado	Funcional	Crítica	Captura histórica «Bucket not found»; pruebas posteriores de configuración y permisos.
Pantalla de inicio sin avanzar	Mensaje / acceso	Alta	Captura y falta de conexión informada. Causa exacta no confirmada con registros del dispositivo.
Salinas y Salinas2 como turnos distintos	Modelo funcional	Alta	El usuario aclaró que son la misma familia; prueba actual de hogar compartido.
Limpieza sin repetición del ciclo	Funcional	Media	Solicitud de repetir el mismo orden semanal; pruebas de rotación.
Interfaz y acciones poco claras	Interfaz / navegación	Media	Observaciones del usuario y comparación de las versiones 4.2 y 4.3.1.

13. Fase 10 — Corregir y volver a probar

Las tres primeras mejoras se pueden contrastar entre la referencia 4.2 y la versión 4.3.1. El error de almacenamiento del QR corresponde a una etapa anterior a 4.2; no se atribuye ese fallo al APK 4.2 adjunto.

Problema / evidencia	Causa probable	Solución implementada	Resultado posterior
Interfaz con flechas y textos poco ajustados; observación del usuario.	Presentación y distribución de controles poco claras.	Botones con acciones escritas, Cerrar, navegación inferior y ajuste de texto.	270 combinaciones de rol, tamaño y letra comprobadas en el desarrollo; 26 vistas verificadas de nuevo. Validación con personas pendiente.
Falta de repetición semanal del orden; solicitud del usuario.	La versión de referencia no disponía de configuración de rotación automática.	Configuración del ciclo y generación semanal; conserva turnos existentes.	42 comprobaciones de interfaz y 41 de rotación SQL pasan. Cron no se ejecuta en PostgreSQL local.
Salinas2 debe compartir limpieza con Salinas; aclaración del usuario.	Se usaban identificadores contables distintos para consultar y autorizar la limpieza.	Alias de hogar solo para limpieza; ambas cuentas usan el mismo turno.	26 comprobaciones SQL de hogar compartido pasan, incluidos permisos y conservación contable.
Carga inicial sin pantalla utilizable; captura histórica.	La interfaz podía quedar oculta si faltaba un script; la conexión también condiciona la carga de cuenta.	Pantalla inicial visible, Reintentar, recuperación de errores y archivos versionados sin caché antigua.	20 comprobaciones de arranque y seis escenarios en Chromium pasan. El bloqueo exacto del teléfono no se da por resuelto sin su confirmación.

Comparación antes / después

Indicador	Antes	Después
Tiempo de tareas de personas	No medido.	Pendiente de medir; no se calcula reducción.
Errores de personas	No contados.	Pendiente de observar con los mismos criterios.
Ayudas de personas	No registradas.	Pendiente de registrar en las cinco tareas.
Éxito de personas	Sin porcentaje.	Pendiente: completadas / intentadas × 100.
Orden semanal	Sin automatización en 4.2.	Rotación comprobada con datos ficticios; cinco hogares en el plan configurado.
Agrupación de limpieza	Identificadores distintos sin alias de hogar.	Una familia de limpieza compartida; conserva separación de pagos.

Reflexión: hay evidencia técnica de nuevas capacidades y de los casos que la versión actual supera. Todavía no hay evidencia suficiente para afirmar que un usuario completa las tareas más rápido o necesita menos ayuda. Esa conclusión requiere realizar la sesión y completar las métricas. Una prueba que pasa demuestra su escenario y sus condiciones, no que la aplicación sea perfecta.

14. Fase 11 — Relación con Kotlin

La ficha solicita Kotlin. En esta aplicación las correcciones se implementaron en Java, JavaScript y SQL. La siguiente tabla explica los conceptos equivalentes que se utilizarían en una implementación Kotlin; no afirma que ese código Kotlin ya exista en el proyecto.

Problema o tarea	Elemento Kotlin	Explicación para la defensa
Validar imagen, nombre y monto	if / when	Comprobar condiciones y rechazar entradas inválidas antes de procesarlas. En el proyecto actual hay condiciones JavaScript y restricciones HTML/SQL.
Mostrar carga, éxito o error	when	Elegir la interfaz según el estado. Equivalente a decidir entre pantalla de inicio, cuenta y mensaje de error.
Calcular saldos	Funciones	Encapsular el cálculo y probar entradas conocidas. Ejemplo comprobado: 100 de deuda menos 40 pagados da 60.
Ordenar familias y tareas	List / MutableList	Representar una colección ordenada. El ciclo usa cinco hogares, aunque existan dos cuentas de uno de ellos.
Mostrar registros de una familia	find / filter	Buscar o seleccionar solo los elementos que corresponden al usuario y a su rol. El permiso del servidor también debe controlarse.
Representar familia, pago o turno	data class	Definir un modelo con atributos y tipos. En el proyecto actual estos datos se reciben como objetos JavaScript y filas SQL.
QR, sesión o dato que puede faltar	Null Safety	Comprobar ausencia y mostrar una alternativa en vez de asumir que el dato siempre existe. En JavaScript se usan comprobaciones y acceso opcional.

15. Uso de IA como tutor

Registro real: se utilizó IA para revisar requisitos, modificar la aplicación, preparar pruebas y elaborar esta ficha. Se debe explicar qué se hizo y sus límites; no se presenta la asistencia como si todas las tareas hubieran sido manuales.

Problema	Solicitud usada en esta conversación	Aprendizaje que se debe explicar	Qué se modificó o preparó
QR y familias	Se pidió corregir el QR y permitir editar nombres y reasignar usuarios.	Cómo se relacionan almacenamiento, permisos, validación e identidad de una familia.	Almacenamiento privado, funciones de administrador y formularios.
Limpieza e interfaz	Se pidió mejorar botones, textos y repetir semanalmente el orden; se aclaró que Salinas y Salinas2 son la misma familia.	Diferencia entre cuenta, familia contable y hogar de limpieza; reglas de un ciclo.	Interfaz, rotación y alias de hogar.
Arranque y ficha	Se comunicó la pantalla en blanco, luego la falta de internet, y se pidió ayuda para llenar la ficha.	Diferenciar una hipótesis de una causa confirmada y una prueba técnica de una observación humana.	Recuperación del arranque e informe con evidencia y pendientes.

Cada integrante debe reescribir los aprendizajes con sus palabras después de revisar el código. No se han inventado prompts de tutoría como si ya se hubieran utilizado ni se afirma que cada integrante domine los cambios sin explicarlos.

16. Productos a entregar

Producto	Material preparado / pendiente
Aplicación inicial y mejorada	Carpeta aplicacion/: APK 4.2 de referencia y APK 4.3.1 mejorado. Los dos son versiones de prueba.
Plan de pruebas	Cinco tareas de usabilidad, diez casos funcionales, siete de validación, tres variables de límites, tres de estrés y tres de persistencia.
Informe de usabilidad	Guion y tablas listos. Faltan participante, dispositivo, observaciones, tiempos, errores, ayudas y éxito.
Problemas y prioridad	Secciones 11 y 12: clasificación y prioridades con evidencia y alcance.
Tres mejoras y nueva prueba	Sección 13: interfaz, rotación y hogar compartido; además, recuperación del arranque.
Evidencia técnica	Logs, JSON, scripts de pruebas locales y capturas de ejemplo en evidencias/.
Registro de IA	Sección 15; cada integrante debe explicar y completar sus aprendizajes.
Defensa	Guía de preguntas de esta página. Confirmar el requisito de Kotlin con el docente, porque el proyecto actual usa Java y JavaScript.

17. Guía para la defensa

¿Qué se probó? Acceso, QR, familias, permisos, limpieza, validación y crecimiento de datos. La sesión con personas queda pendiente.

¿Qué problemas se encontraron? QR sin almacenamiento en una etapa inicial, limpieza sin repetición ni agrupación de Salinas e interfaz poco clara. En el teléfono se observó bloqueo sin internet; su causa exacta no se confirmó.

¿Qué se modificó? Interfaz, administración, almacenamiento, rotación, hogar compartido y recuperación del arranque.

¿Cómo se comprobó? Con los casos y logs adjuntos. Los milisegundos del renderizado no son tiempos de tareas de personas. Para evaluar usabilidad falta medir a participantes.

¿Qué relación tiene con Kotlin? Condiciones, funciones, colecciones, modelos y datos ausentes. Son equivalencias conceptuales: el proyecto actual no está escrito en Kotlin.

18. Bibliografía y fuentes

Ficha de Trabajo 02: Evaluación de una aplicación móvil: usabilidad, funcionalidad, límites y estrés. Documento proporcionado para esta actividad, seis páginas.

Android Developers: Fundamentals of testing Android apps — https://developer.android.com/training/testing/fundamentals

Android Developers: Automate UI tests — https://developer.android.com/training/testing/instrumented-tests/ui-tests

Kotlin Documentation: Null safety — https://kotlinlang.org/docs/null-safety.html

ISTQB: CTFL y materiales de pruebas de usabilidad — https://www.istqb.org. ISO/IEC 25010:2023: modelo de calidad de producto, referencia indicada en la ficha.

19. Cierre

El objetivo es demostrar que el equipo sabe probar, observar, corregir y justificar. Este informe queda prellenado con evidencia técnica verificable. Completar la prueba con un compañero y las tres reaperturas del APK antes de presentarlo como evaluación terminada.

Anexo — Evidencias y reproducción

Archivo / grupo	Qué demuestra y qué no demuestra
pruebas-interfaz.txt	20 comprobaciones de arranque; 11 de autenticación; Restaurante; 36 de administración; 42 de rotación; 26 vistas de demostración. No son pruebas con personas.
pruebas-base-datos.txt	59 comprobaciones de backend, 41 de rotación y 26 de hogar compartido: 126 en total. Base desechable; Cron no se prueba en esa imagen.
pruebas-arranque-navegador.txt	Seis escenarios de arranque con el SDK incluido. La recuperación conserva almacenamiento local. No reproduce todos los teléfonos Android.
limites-repeticiones.json y .cjs	Cuatro controles del tamaño QR, 20 renombrados sucesivos y 20 intentos de guardado simultáneo. Backend en memoria.
pruebas-limites-sql.txt y limites.sql	Ocho comprobaciones nuevas de longitudes en nombre y tarea; límites y rechazo fuera de rango. Solo para base desechable.
estres-datos.json y .py	Tres mediciones por nivel: 100, 500 y 1.000 pagos; Chromium de escritorio, datos ficticios, viewport 390 × 844.
validacion-monto.json	Cinco entradas contra restricciones HTML del monto. No demuestra validación de la RPC de pagos.
alcance.json	Identificación de la versión, commit y límites de la evidencia.
Capturas de ejemplo	Presentan la interfaz con datos simulados; no contienen resultados de participantes ni comprobantes bancarios reales.

Código evaluado: commit 54c7c2dba39e6855207856cb769ee19b6f1d9515 del repositorio achaalex326-ux/Casa. Los tests del proyecto están en CasaFamiliasAndroid/validacion. La ejecución de esta ficha no cambió el código de la aplicación.

Para repetir la suite de interfaz: con Node y las dependencias del proyecto, ejecutar npm test desde CasaFamiliasAndroid/validacion. Para la base aislada: con Docker, ejecutar python test_backend.py en esa carpeta. Los scripts adicionales tienen rutas del entorno de elaboración; adaptar las rutas si se ejecutan en otra computadora. No ejecutar fixtures, limites.sql ni comprobaciones de prueba en Supabase real.

Antes de entregar: completar lo que requiere observación

1. Hacer las cinco tareas con un compañero sin guiarlo. 2. Anotar tiempos, errores, ayudas y éxito. 3. Cerrar y reabrir el APK en las tres pruebas de persistencia. 4. Comparar versiones con las mismas condiciones. 5. Agregar las capturas propias de esas pruebas y revisar el requisito de Kotlin con el docente.